AUTHORITATIVE OPERATING SEQUENCE

Quantara
New Prop Account / New Broker Deployment Runbook

Corrected Phase C / Phase D ordering, exact pre-C5 full-suite gate, hybrid-routing invariants, verified 103-edge D4 FAST promotion, and fail-closed release validation

Purpose
Use this runbook when moving Quantara to a new prop account, broker, MT5 server/feed, or when the allocator produces a different selected-edge universe. It separates account/broker adaptation from model authority, and separates Phase-D optimization from the C4 authoritative trading path.

Reliability note
No operating document can guarantee that future source-code defects will never exist. This revision is designed to eliminate the known ordering mistakes and to surface repository/test incompatibilities before expensive certification or live arming.

Repository basis: Wells21/trading_models, branch v2, root trading_models_v2. All commands below are standardized to run from the trading_models_v2 root unless explicitly stated otherwise.

Revision date: 22 August 2026 - Revision 5

Status: corrected against the current repository implementation and the Moneta 103-edge migration. Revision 5 incorporates the completed AUTHORITATIVE-to-FAST D4 promotion, 103/103 consolidated real-history certification, production certificate promotion, focused overlay/router/registry closure, final production-hardening PASS, and the required new-account runtime-state archive procedure.

Golden safety rule
During migration/certification, the dashboard and executor stay stopped. Build a REAL_LIVE manifest if required, but keep PORTFOLIO_EXECUTION_MODE=paper and ENABLE_ORDER_SEND=False until the final dashboard-only verification passes.

1. What Was Wrong in the Previous Runbook

The previous document contained individually reasonable instructions placed in an incompatible order. That ordering is what produced the current certification failures.

Previous problem	Correct interpretation
C5 was run while stale D3/D4 activation existed	C5 intentionally requires C4 activation to be absent. P0 imports the dashboard during C5; if an old D3 activation remains, D3 tries to install and immediately demands active C4, causing "D.3 requires active certified C.4 routing".
D1 refresh was placed too late	C5 runs the full unit suite. The Phase-D tests require D1 to match the current C3/live-manifest selected-edge universe. Therefore D1 must be refreshed before C5 whenever the selected edge set changes.
The document contradicted itself about D2/D3	A changed universe does not mean all new edges must be put through the giant D2/D3/D4 optimization certifications. D3/D4 are optional acceleration layers. Existing certified edges can remain optimized; new/uncertified current edges trade through C4 authority.
Dashboard validation order was backwards	A dashboard status file should be judged only after the dashboard has been started by itself and has produced a fresh current-state status. The executor is armed only after that check passes.
Focused tests were mistaken for the C5 test gate	C5 runs unittest discovery across the entire tests directory. Passing P0, hardening, and two Phase-D files is not equivalent to passing C5's full test suite. The exact discovery command must run before C5.
A stale D4 unit test still assumed every selected edge must be FAST	Hybrid D4 intentionally permits a mixed portfolio: FAST_CONTINUATION for edge-level D4-certified/provider-capable edges and AUTHORITATIVE_BATCH for the rest. The correct invariant is FAST + AUTHORITATIVE = SELECTED and unsupported_trading_edges = 0. A test requiring FAST = SELECTED is stale and must be corrected before C5 can seal.

> **Correct dependency chain**
> New live manifest -> fresh broker bar policy -> (if universe changed) rebuild C1/C2/C3 -> refresh D1 + inert D4.1 -> C5 with C4/D3/D4 activations absent -> validate C5 -> activate C4 -> regenerate current D3 hybrid activation -> validate/enable D4 overlay -> continuation health -> dashboard-only verification -> executor.

2. Operating Rules That Apply to Every Migration

- Run every command from C:\Users\Administrator\Desktop\Trading Models\trading_models_v2 unless a command changes directory explicitly.
- Stop immediately when a required script exits non-zero. Do not continue merely because a later stage might repair it.
- Do not copy real broker credentials into live_settings.py or Git. Keep them only in portfolio_live\local_live_settings.json.
- Do not change model features, thresholds, AFML logic, edge geometry, or research rules as a way to make a broker migration pass. Broker/account adaptation belongs downstream.
- Do not force new current edges into FAST Phase-D routing. AUTHORITATIVE C4 routing is a supported production mode, not a failure.
- Do not restore an old D3 activation blindly after the selected universe changes. Rebuild the activation payload against the current universe.
- Never start the executor during C5 or while activation files are intentionally removed.
- Treat the exact full unittest discovery used by C5 as a mandatory pre-C5 gate. Focused tests are diagnostic only, and any intentionally failing negative-control script under test_*.py must be import-safe.

```
Set-Location "C:\Users\Administrator\Desktop\Trading Models\trading_models_v2"
```

3. Configure the New Prop Account Rules

Authoritative account-rule file:

```
portfolio_allocator\config\active_account_rules.json
```

Scripts 08-11 and the live system consume the same normalized account rules. Verify every rule against the prop firm/account documentation; do not infer an HWM or reset rule.

[ ] profile_label and account_stage
[ ] initial_balance, current_balance, current_high_watermark
[ ] profit target: enabled, pct/amount, and stop behavior
[ ] overall drawdown: type, pct/amount, breach basis, HWM basis when trailing, and whether the trail stops at initial balance
[ ] daily drawdown: pct/amount, calculation mode, breach basis, reset timezone/hour
[ ] minimum trading days and any minimum-day-profit rule
[ ] maximum risk per trade if the prop imposes one
[ ] daily symbol risk if applicable
[ ] floating-loss guard if applicable
[ ] news-trading, weekend-holding, automation, and VPS permissions

> **Fresh challenge vs account in progress**
> For a genuinely fresh challenge, initial_balance, current_balance, and current_high_watermark normally begin at the account starting capital. For an account already in progress, use the real current balance/HWM. Never reset or invent the HWM just to obtain a better allocator result.

4. Configure local_live_settings.json in Safe Preflight Mode

Real credentials and broker-specific overrides belong only here:

```
portfolio_live\local_live_settings.json
```

Update the target-account identity and connection values:

- mt5_login
- mt5_password
- mt5_server
- mt5_path
- mt5_symbol_map
- portfolio_expected_account_login
- portfolio_expected_account_server
- portfolio_expected_account_currency
- portfolio_account_stage if you use the optional local cross-check

Recommended preflight arming configuration:

```
{
  "live_mode": "REAL_LIVE",
  "allow_real_live_trading_export": true,
  "portfolio_execution_mode": "paper",
  "enable_order_send": false
}
```

Why this is correct: the manifest builder requires REAL_LIVE plus allow_real_live_trading_export=true to produce LIVE_APPROVED/live_ready/live_trade_enabled. Actual order submission is separately gated by execution mode and ENABLE_ORDER_SEND. This lets you certify a real-live deployment while remaining unable to send an order.

5. Configure Symbols and Execution Guards

Populate mt5_symbol_map for the broker symbols actually needed by the selected universe. Example logical assets: XAUUSD, BTCUSD, NASDAQ, DAX, GBPJPY, JPN, USOIL, US500, HK, USDMXN. The broker names may differ.

Every selected asset must have an explicit execution guard with all four fields:

- max_spread_points
- entry_drift_limit_price
- deviation_points
- max_lot_cap

The current code contains defaults for the ten current logical assets, including HK and USDMXN. A genuinely new Quantara asset requires a real guard definition before live environment validation can pass.

6. Update Broker Symbol Seeds, Then Connect the Intended MT5 Account

Editable seed:

```
portfolio_allocator\broker_contract_specs.json
```

Treat this as a symbol/alias seed only. 00A connects to MT5 and captures the authoritative current broker contract snapshot. Local mt5_symbol_map takes precedence for symbol resolution.

[ ] Open the intended MT5 terminal.

[ ] Log in to the intended new prop account and server.

[ ] Confirm Market Watch and tick data are available for mapped symbols.

[ ] Keep Quantara dashboard and executor stopped.

7. Run the Full Allocator for a New Broker / Prop Account

> **Full run required for a broker change**
> Even when the logical asset count is unchanged, a different broker/feed requires 00A/00B and the full downstream allocator because contract size, tick size/value, min lot, lot step, max lot, profit currency, order_calc_profit behavior and historical currency conversion affect executable risk and selection.

```
python .\portfolio_allocator\00a_capture_mt5_broker_profile.py
python .\portfolio_allocator\00b_build_historical_conversion_artifacts.py
python .\portfolio_allocator\01_ingest_pass_edge_manifests.py
python .\portfolio_allocator\02_build_edge_candidate_bank.py
python .\portfolio_allocator\03_classify_edge_candidate_bank.py
python .\portfolio_allocator\04b_build_threshold_grid_trade_ledger.py
python .\portfolio_allocator\04bb_validate_geometry_handoff.py
python .\portfolio_allocator\04c_select_representative_threshold_candidates.py
python .\portfolio_allocator\04ca_analyze_rejected_edge_gates.py
python .\portfolio_allocator\05_build_representative_trade_ledger.py
python .\portfolio_allocator\06_build_representative_return_streams.py
python .\portfolio_allocator\07_hrp_linkage_robustness.py
python .\portfolio_allocator\07b_validate_geometry_handoff.py
python .\portfolio_allocator\08_rolling_start_prop_challenge_optimizer.py --workers 7
python .\portfolio_allocator\08b_pbo_cscv_validation.py
python .\portfolio_allocator\08c_joint_risk_throttle_grid_search.py --workers 7
python .\portfolio_allocator\09_fast_manual_variant_lab.py --quick-probe --workers 7
python .\portfolio_allocator\09_fast_manual_variant_lab.py --full-single --workers 7
python .\portfolio_allocator\09_fast_manual_variant_lab.py --full-single --workers 7 --accept-moderate-hard-fail  (only if the line above reports FAIL with hard-fail rate <= 3% at every cost tier -- see the callout below)
python .\portfolio_allocator\10_final_candidate_walkthrough.py --workers 7  [add --accept-moderate-hard-fail if Step 09 needed it]
python .\portfolio_allocator\11_monte_carlo_stress_validation.py --workers 7  [add --accept-moderate-hard-fail if Step 09/10 needed it]
```

New in this revision: Steps 08b/08c and the moderate-hard-fail / manual-override flags

08b_pbo_cscv_validation.py runs an AFML Ch.11 CSCV probability-of-backtest-overfitting check on Step 08's scenario grid. It is diagnostic only (reports a PBO percentage); it does not gate or change Step 08's selection.

08c_joint_risk_throttle_grid_search.py searches Step 09's guard/reserve combinations jointly against Step 08's candidates, because the best risk-budget/single-trade-cap combination without a throttle is not always the best once a throttle and admission reserve are layered on top. Run it with no arguments for the default unattended two-stage search (a cheap survey across every candidate and guard, then a full-density confirmation of the most promising pairs); it writes best_joint_scenario.csv and best_joint_variant_spec.json into the Step 08 output directory, which 09_fast_manual_variant_lab.py then loads automatically — no manual edit of ACTIVE_VARIANT_SPEC is required. Pass --manual together with --candidates/--guards/--reserve-pcts/--reserve-cost-allowances for the original hand-scoped cross-product mode.

--workers N is available on 08, 08c, 09, 10 and 11 (default: CPU count minus 1) to parallelize the rolling-start simulation grid across processes. Pass --workers 1 to force the original single-process path.

--accept-moderate-hard-fail is available independently on 09, 10 and 11. By default each of those three scripts requires a literal 0% hard-fail rate before approving a candidate; this flag instead accepts a candidate whose hard-fail rate is nonzero but stays ≤ 3% at every tested cost-stress tier (0.00R/0.02R/0.05R), with zero daily-drawdown failures and no reserve-contract violations — a deliberate, explicitly logged trade of a small amount of historical tail risk for meaningfully faster challenge completion. It does NOT carry over automatically between scripts: pass it again at 10 and 11 if 09 needed it, or the later script will legitimately reject a candidate the earlier one approved.

09_fast_manual_variant_lab.py also accepts --manual-step08-candidate <foundation_candidate_name>, --manual-guard-profile <name>, --manual-reserve-mode {off,static,dynamic_projected_open_risk}, --manual-reserve-pct <float> and --manual-reserve-cost-allowance <float>, so a human reviewing 08c's full joint_grid_ranked.csv can send a different Step 08 candidate and/or guard/reserve combination through the same validated 09→10→11 pipeline instead of the automated pick. The candidate override and the guard/reserve override are independent — pass one, the other, both, or neither.

Allocator gate

Do not build the live manifest unless Script 11 ends in STRESS_PASS and produces the current stress-validated live configuration for the active account rules.

8. Build and Validate the New Live Manifest

```
python .\portfolio_live\00_build_live_deployment_manifest.py
python .\portfolio_live\01_validate_live_manifest.py
python .\portfolio_live\02_validate_live_environment.py --require-mt5
```

Required live outcome before routing certification:

[ ] deployment_status = LIVE_APPROVED
[ ] live_ready = True
[ ] live_trade_enabled = True
[ ] stress_validation_status = STRESS_PASS
[ ] current account rules match
[ ] selected assets have valid symbol mappings and execution guards

9. New Broker / Feed: Rebuild the Live Bar Policy

Live bar thresholds are feed/tick-flow dependent. Never carry the previous broker policy forward blindly.

```
python .\portfolio_live\03_live_bar_threshold_diagnostic.py --fresh
python .\portfolio_live\04_freeze_live_bar_thresholds.py --require-pass
```

Required gate

03 must report PASS for every current selected asset and 04 must report POLICY_PASS / policy_ready=True.

10. Decide Whether the Selected Edge Universe Changed

Compare the new live manifest to the existing frozen C3 consumer authority. This decides whether Phase C must be rebuilt.

```
@'
import json
from pathlib import Path

root = Path.cwd()
manifest = json.loads((root / "portfolio_live/live_manifest.json").read_text(encoding="utf-8-sig"))
c3_path = root / "portfolio_live/config/phase_c/consumer_contract.json"

live_ids = sorted(str(x["edge_id"]) for x in manifest.get("selected_edges", []) if x.get("edge_id"))
if not c3_path.is_file():
    print("PHASE_C_REBUILD_REQUIRED: consumer_contract.json missing")
    raise SystemExit(0)

c3 = json.loads(c3_path.read_text(encoding="utf-8-sig"))
c3_ids = sorted(str(x["edge_id"]) for x in c3.get("edges", []) if x.get("edge_id"))

print("live edges:", len(live_ids))
print("C3 edges  :", len(c3_ids))
print("added     :", sorted(set(live_ids) - set(c3_ids)))
print("removed   :", sorted(set(c3_ids) - set(live_ids)))
print("UNCHANGED :", live_ids == c3_ids)
'@ | python -
```

> **Branch rule**
> If UNCHANGED=True, follow Part 11A. If false - or if the read-only C4 structural check fails - follow Part 11B.

11A. Selected Edge Set Unchanged

A broker/account change alone does not require rebuilding C1/C2/C3 or rerunning heavy Phase-D certifications when the structural selected universe remains valid.

```
python .\portfolio_live\phase_c_04_prepare_activation.py
```

You want Static status: PASS and Candidate status: PASS. This command is read-only and writes no activation.

Then run the hybrid Phase-D validation in Part 15. If C4/D3/D4 all validate against the current manifest, keep the existing optimization certifications. Continue to continuation health and dashboard-only preflight.

> **Escalation**
> If the C4 read-only check fails because the selected assets/edges or frozen C1/C2/C3 authorities no longer match current state, do not patch around it. Switch to Part 11B.

11B. Selected Edge Set Changed (or Phase C Authority Is Stale)

This is the path that was misplaced in the old runbook. The activation-removal window must begin before certification, and D1 must become current before C5 runs its full unit suite.

11B.1 Back Up and Remove C4, D3 and D4 Activations

```
$stamp = Get-Date -Format "yyyyMMdd_HHmmss"
$backup = Join-Path $env:TEMP "quantara_activation_backup_$stamp"
New-Item -ItemType Directory -Force $backup | Out-Null

$files = @(
  ".\portfolio_live\config\phase_c\production_routing_activation.json",
  ".\portfolio_live\config\phase_d\production_routing_activation.json",
  ".\portfolio_live\config\phase_d\provider_overlay_activation.json"
)

foreach ($f in $files) {
```

```
  if (Test-Path $f) {
    Move-Item $f (Join-Path $backup ([IO.Path]::GetFileName($f))) -Force
  }
}

"Activation backup: $backup"
$files | ForEach-Object { "$_ exists = $(Test-Path $_)" }
```

All three final existence checks should be False during the C5 certification window.

> **Why all three?**
> C5 itself requires C4 activation absent. P0 imports the dashboard. An old D3 activation would then attempt to install on top of inactive C4 and fail. D4 is downstream of D3, so its activation should also be absent during this window.

11B.2 Rebuild Current-Universe Phase C Authorities

```
python .\portfolio_live\phase_c_01_compile_feature_contract.py
python .\portfolio_live\phase_c_01_validate_feature_contract.py
python .\portfolio_live\phase_c_02_certify_incremental_features.py
python .\portfolio_live\phase_c_03_compile_consumer_contract.py
python .\portfolio_live\phase_c_03_validate_consumer_contract.py
python .\portfolio_live\phase_c_03_certify_consumer_contract.py
```

Every stage must PASS. C1/C2/C3 are current-universe numerical/consumer authorities; they are not account-risk optimizers.

11B.3 Refresh D1 BEFORE C5

```
python .\portfolio_live\phase_d_01_compile_candidate_contract.py
```

D1 is offline inventory/contract compilation only. It does not enable Phase D routing. Its Assets/Edges counts must exactly match the current live manifest/C3 universe.

> This ordering fixes the D1-currentness failure class
> C5 runs full unittest discovery. test_phase_d_candidate_first requires D1 and C3 to match; test_phase_d_incremental_candidate_contract requires the current D1 to cover the live manifest exactly. Therefore D1 cannot be postponed until after C5. The exact full suite in Part 12.3 is still mandatory.

11B.4 Refresh the Inert D4.1 Current Contract

```
python .\portfolio_live\phase_d_04_compile_incremental_candidate_contract.py --require-pass
```

This is not the giant D4 real-history certifier. It is an inert source-signature/current-inventory compiler and does not activate provider routing.

12. Pre-C5 Repository Invariants and the Exact C5 Test Gate

Before spending another full C5 run, make the inexpensive/current-contract checks pass first.

12.1 Verify the signal-validity production contract

```
Select-String -Path .\portfolio_live\live_settings.py -Pattern "PORTFOLIO_(DASHBOARD|EXECUTION)_MAX_SIGNAL_AGE_SECONDS"
```

Run the non-mutating source check and the production-hardening validator. For the current contract both dashboard and executor signal-age values must resolve to 120 seconds (changed from 1000 seconds on 2026-09-10, commit a65327bb “Task 1 patcher done” — this is the current, deliberate contract, not a regression; do not restore 1000 seconds). If the hardening validator already reports PASS, do not edit these values merely because an older runbook mentioned 300 or 1000 seconds.

Do not mutate a passing contract
If validate_production_hardening_patch.py reports PASS, leave the signal-age source alone. If it fails on signal age, correct the tracked source mismatch through the repository workflow; do not hide it with an account-specific override.

```
python .\portfolio_live\validate_production_hardening_patch.py
```

12.2 Run focused validators with activations still absent

```
python .\portfolio_live\validate_production_hardening_patch.py
python .\portfolio_live\validate_p0_production_patch.py
python .\tests\test_phase_d_candidate_first.py
python .\tests\test_phase_d_incremental_candidate_contract.py
```

These focused checks must PASS, but they are not the C5 unit-test gate. Their purpose is to localize common migration defects cheaply before the exact full-suite run in Part 12.3.

12.3 Run the exact full unittest discovery that C5 runs

This is mandatory. C5 executes this same discovery command internally. Passing only selected test files does not prove that C5 can seal.

```
python -m unittest discover -s .\tests -p "test_*.py" -v
```

Required result: the command exits 0 and ends with OK. If it fails, do not run C5 yet. Fix or reconcile the failing test/current-state contract first.

If C5 has already failed full_unittest_discovery, do not rerun C5 merely to see the error. Extract the captured unittest tail from the candidate report:

```
$r = Get-Content .\portfolio_live\config\phase_c\phase_c5_candidate_report.json -Raw | ConvertFrom-Json
$r.evidence.full_unittest_discovery.output_tail
```

12.4 Hybrid D4 invariant - do not require every selected edge to be FAST

The D4.10 architecture is intentionally hybrid. A selected edge may run FAST_CONTINUATION only when its current D1 candidate/primary signatures have provider coverage and the production D4 real-history certificate covers that exact edge. Otherwise it remains fully tradable through AUTHORITATIVE_BATCH. The invariant is complete, disjoint FAST + AUTHORITATIVE coverage with unsupported_trading_edges = 0.

Resolved repository trap #1: the current-state overlay assertion must never hard-code FAST = selected or AUTHORITATIVE = 0. The Moneta working tree now uses the hybrid-safe invariant. A separate integrity test, test_archived_v3_certifier_is_exact, intentionally pins the exact certifier SHA. When the certifier is deliberately and safely changed, verify the new certifier bytes first and then re-pin that exact SHA; never delete the integrity test or make it auto-accept arbitrary current bytes.

```
Correct test logic (conceptual)
selected = fast + authoritative
unsupported_trading_edges = 0
# Do NOT assert authoritative == 0 unless the current universe is actually fully D4-certified.
```

Wrong stale invariant	FAST_CONTINUATION == selected_edge_count and AUTHORITATIVE_BATCH == 0
Correct production invariant	FAST_CONTINUATION + AUTHORITATIVE_BATCH == selected_edge_count
Hard safety invariant	unsupported_trading_edges == 0

12.5 Intentional negative controls must be import-safe

Resolved repository trap #2 / future rule: an intentional must-fail negative-control module must be import-safe under unittest discovery. The volatilitycarry control should preserve its standalone non-zero behavior, but its executable

mutation/assertion belongs behind main() and if __name__ == "__main__" so C5 discovery can import it without executing the intentional failure.

Correct contract: preserve the standalone must-fail behavior, but place the executable negative-control body inside main() and invoke it only under if __name__ == "__main__". Unittest discovery must be able to import the module successfully. Do not delete the negative control, weaken its parity assertion, or make its standalone execution return zero.

Future rule: no intentional failure/negative-control diagnostic may execute at import time under a filename matched by C5's test_*.py discovery pattern. Either make it import-safe with a main guard or keep it outside the discovery pattern.

13. Seal Phase C.5, Validate It, Then Activate C4

C4/D3/D4 activation files must still be absent when C5 starts.

```
python .\portfolio_live\phase_c_05_certify_production_routing.py --seal-production
python .\portfolio_live\phase_c_05_validate_production_certificate.py
```

C5 required gates must all PASS: C4 static contract, C4 self-test, frozen C1, frozen C2, frozen C3, all-selected-edge model/decision parity, P0 validator, production hardening validator, and full unittest discovery. Part 12.3 should already have proven the last gate before this long run begins.

> **C5 does not activate C4**
> A successful C5 writes the production certificate and intentionally leaves activation absent. That separation is by design.

Now activate C4:

```
python .\portfolio_live\phase_c_04_prepare_activation.py `
  --activate `
  --i-understand-this-enables-phase-c4-production-routing
```

The activation script refuses to write unless the sealed C5 certificate is present and valid.

14. Rebuild Current Hybrid Phase-D Activations WITHOUT Giant Recertification

> **Core scalability rule**
> D2/D3/D4 certifications are acceleration authorities, not prerequisites for an edge to trade. Current selected edges that lack old Phase-D certificates remain tradeable through C4 authoritative routing. unsupported_trading_edges must remain 0.

14.1 Heavy Phase-D recertification is optional for acceleration, not required for tradability

- phase_d_02_certify_candidate_first.py
- phase_d_03_certify_production_router.py --seal-production
- the large D4 real-history provider certification merely because the broker/current universe changed

A broker/account migration does not require these heavy jobs merely to make new current edges tradable. If you deliberately rerun D2/D3 for the whole current universe and they PASS, that is valid additional acceleration evidence; it is simply not a prerequisite for C4-authoritative tradability. The large D4 real-history provider certification remains separate and should not be rerun just to eliminate legitimate AUTHORITATIVE edges.

14.2 Generate a fresh current D3 hybrid activation

Run this only AFTER C4 has been activated. It reuses the retained PASS D2/D3 certified subset, intersects it with the current selected universe, and leaves the remainder on C4 authority.

```
@'
import json
import sys
from pathlib import Path

root = Path.cwd().resolve()
live = root / "portfolio_live"
```

```
if str(live) not in sys.path:
    sys.path.insert(0, str(live))

import phase_d_03_production_router as d3

try:
    payload = d3.build_activation_payload(v2_root=root)
except Exception as exc:
    print("D3 HYBRID ACTIVATION NOT AVAILABLE:", type(exc).__name__, exc)
    print("Leave D3/D4 inactive; current edges remain on certified C4 authority.")
    raise SystemExit(2)

path = live / "config" / "phase_d" / "production_routing_activation.json"
path.parent.mkdir(parents=True, exist_ok=True)
path.write_text(json.dumps(payload, indent=2, sort_keys=True) + "\n", encoding="utf-8")

v = d3.validate_activation(payload, v2_root=root)
print("D3 activation:", v.get("status"))
print("D3 optimized :", v.get("optimized_edge_count"))
print("D3 C4 auth   :", v.get("authoritative_c4_edge_count"))
print("D3 failures  :", v.get("failures"))
if v.get("status") != "PASS":
    raise SystemExit(1)
'@ | python -
```

If no certified overlap remains

Do not force a D3 activation. Leave D3/D4 inactive and run C4 authority only. That is safer than manufacturing a certification.

14.3 Create/validate the D4 overlay activation only when D3 hybrid is valid

D4 partitions the current universe at runtime: edge-level real-history-certified/provider-capable edges are FAST_CONTINUATION; all other current edges are AUTHORITATIVE_BATCH. The activation file is intentionally minimal: schema_version, status=PASS, and mode=FAST_CONTINUATION. It does not pin a certificate SHA, edge count, or provider count. Therefore a valid existing D4 activation does not need regeneration merely because a newly staged PASS production certificate promotes more current edges to FAST; validate_activation() and static_overlay_prerequisites() re-evaluate the current structural state dynamically.

```
@'
import json
import sys
from pathlib import Path

root = Path.cwd().resolve()
live = root / "portfolio_live"
if str(live) not in sys.path:
    sys.path.insert(0, str(live))

import phase_d_04_provider_overlay as d4

payload = {
    "schema_version": d4.D4_ACTIVATION_SCHEMA_VERSION,
    "status": "PASS",
    "mode": d4.FAST_CONTINUATION,
}

v = d4.validate_activation(payload, root)
print("D4 activation:", v.get("status"))
print("D4 failures  :", v.get("failures"))
print("D4 capability:", (v.get("prerequisites") or {}).get("capability_summary"))
if v.get("status") != "PASS":
    raise SystemExit(1)

path = live / "config" / "phase_d" / "provider_overlay_activation.json"
path.write_text(json.dumps(payload, indent=2, sort_keys=True) + "\n", encoding="utf-8")
print("D4 activation written:", path)
'@ | python -
```

Creating the activation does not certify uncovered edges. It enables only the currently certified/provider-capable overlay. If an already-valid FAST_CONTINUATION activation exists, keep it through a certificate-only promotion unless its own schema, status, or mode changes. The production certificate and current D1/provider state determine the partition.

15. Validate the Hybrid C4 / D3 / D4 Routing Before Runtime

```
@'
import json
import sys
from pathlib import Path

root = Path.cwd().resolve()
live = root / "portfolio_live"
if str(live) not in sys.path:
    sys.path.insert(0, str(live))

import phase_c_04_production_router as c4
import phase_d_03_production_router as d3
import phase_d_04_provider_overlay as d4

print("=" * 100)
print("CURRENT HYBRID PRODUCTION ROUTING")
print("=" * 100)

c4_static = c4.static_contract_report(root, force_refresh=True)
print("C4 static            :", c4_static.get("status"))
print("C4 selected assets   :", len(c4_static.get("selected_assets") or []))
print("C4 selected edges    :", len(c4_static.get("selected_edge_ids") or []))
print("C4 failures          :", c4_static.get("failures"))
if c4_static.get("status") != "PASS":
    raise SystemExit("FAIL: C4 current authority invalid")

c4_path = live / "config" / "phase_c" / "production_routing_activation.json"
if not c4_path.is_file():
    raise SystemExit("FAIL: C4 activation missing")

d3_path = live / "config" / "phase_d" / "production_routing_activation.json"
if d3_path.is_file():
    a3 = json.loads(d3_path.read_text(encoding="utf-8-sig"))
    v3 = d3.validate_activation(a3, v2_root=root)
    print("D3 activation        :", v3.get("status"))
    print("D3 optimized edges   :", v3.get("optimized_edge_count"))
    print("D3 C4-authoritative  :", v3.get("authoritative_c4_edge_count"))
    print("D3 failures          :", v3.get("failures"))
    if v3.get("status") != "PASS":
        raise SystemExit("FAIL: D3 activation invalid")
else:
    print("D3 activation        : ABSENT (C4-only mode)")

r4 = d4.static_overlay_prerequisites(root)
print("D4 prerequisites     :", r4.get("status"))
print("D4 selected edges    :", r4.get("selected_edge_count"))
print("D4 FAST eligible     :", r4.get("fast_eligible_edge_count"))
print("D4 authoritative     :", r4.get("authoritative_edge_count"))
print("D4 capability        :", r4.get("capability_summary"))
print("D4 failures          :", r4.get("failures"))

if d3_path.is_file():
    d4_path = live / "config" / "phase_d" / "provider_overlay_activation.json"
    if not d4_path.is_file():
        raise SystemExit("FAIL: D4 activation missing while D3 hybrid is enabled")
    a4 = json.loads(d4_path.read_text(encoding="utf-8-sig"))
    v4 = d4.validate_activation(a4, root)
    print("D4 activation        :", v4.get("status"))
    print("D4 failures          :", v4.get("failures"))
    if v4.get("status") != "PASS":
        raise SystemExit("FAIL: D4 activation invalid")

if r4.get("status") != "PASS":
    raise SystemExit("FAIL: D4 current prerequisites invalid")
if int((r4.get("capability_summary") or {}).get("unsupported_trading_edges") or 0) != 0:
    raise SystemExit("FAIL: unsupported trading edges are non-zero")

print("HYBRID ROUTING VALIDATION: PASS")
'@ | python -
```

> **Interpret the counts correctly**
> Seeing AUTHORITATIVE edges is healthy when the current universe contains edges that were never covered by the old D4 real-history certification. The hard requirement is current authority PASS and unsupported_trading_edges=0 - not that every selected edge must be FAST.

Operational note (2026-09-20): the D.4 consolidated real-history certification goes stale — dropping to 0 FAST-eligible edges, everything safely on AUTHORITATIVE_BATCH — every time the selected edge universe changes, even if every provider's code is already correct and current. This is expected, not a bug: check portfolio_live/phase_d_04_provider_overlay.py's static_overlay_prerequisites()['fast_eligible_edge_count'] after any universe change, and treat 0 as benign (tradability is unaffected) unless you are deliberately pursuing the acceleration in Section 15A below.

Two audit tools exist to catch this drift class immediately instead of as a wall of unrelated-looking test failures: portfolio_live/phase_d_05_audit_provider_signature_drift.py (candidate/primary-side D1 signature drift — run right after phase_d_01_compile_candidate_contract.py or the D.4.1 incremental contract recompile) and shared/validation/audit_payoff_context_id_drift.py (payoff_context_id hash drift in saved threshold/payoff artifacts — run whenever shared/validation/v3_semantic_contract.py's identity-hash logic changes, or if EdgeScorer.validate_artifacts() starts failing with “Threshold/payoff semantic identity mismatch” for edges whose numbers have not actually changed). Both report-only by default; the payoff-context tool takes --fix to apply its safe re-stamp, and refuses to touch any edge where a real data field — not just the identity hash — differs.

15A. Optional AUTHORITATIVE -> FAST Provider Promotion (Parity-Preserving)

Use this only when you deliberately want additional Phase-D acceleration after complete C4/D3 tradability already exists. Do not promote an edge to FAST merely to make a migration pass, and do not change research parameters, model inputs, thresholds, AFML, allocator decisions, or edge geometry to obtain a FAST result.

> **Promotion safety rule**
> Provider implementation is an execution-path optimization. Research/live parity remains the authority. Provider availability alone is never enough: production FAST eligibility also requires edge-level consolidated real-history certification. If any post-promotion structural check fails, restore the previous production certificate and keep the edge authoritative.

15A.1 Inventory the current D1 signatures and provider coverage

Before writing provider code, inventory each selected edge against D1 candidate_logic_signature and primary_logic_signature, the dynamically discovered candidate/primary provider registries, and the current production D4 edge certificate. Classify each edge as already FAST, provider-capable but certificate-only, missing candidate/primary provider coverage, or unsupported. The required pre-change invariant is still unsupported_trading_edges = 0; AUTHORITATIVE edges are healthy during the work.

- Do not create per-asset provider registries or recipe tables. A source-equivalent edge with the same exact D1 logic signature should bind to the same provider family automatically; a genuinely new signature remains unbound until implemented and certified.

15A.2 Implement only the missing exact-signature provider families

Implement candidate and primary providers against the exact D1 signatures. Preserve causal ordering, event spacing, state updates, constants, side semantics, and source behavior. Dynamic registry discovery must remain fail-closed on duplicate signature claims. No provider implementation may add result-driven filters or alter the research definition.

- Install provider code first while leaving the existing production D4 certificate unchanged. This ensures new code cannot silently become production FAST before the real-history parity certificate is produced.

15A.3 Run a full staged consolidated real-history certificate

Write the new report to a separate STAGING path. Do not overwrite the fixed production certificate during certification. The certifier --repo-root is the repository parent that contains trading_models_v2, not the V2 root itself.

```
python .\portfolio_live\certification\phase_d4_consolidated_real_history_certifier_v3.py `
  --repo-root "C:\Users\Administrator\Desktop\Trading Models" `
  --output "C:\Users\Administrator\Desktop\Trading
```

```
Models\trading_models_v2\portfolio_live\outputs\phase_d4_provider_real_history_certification_v2_STAGING_YYYYMMDD.json"
```

Required release evidence: report status PASS; edges_checked = current selected count; edges_passed = current selected count; edges_failed = 0; unsupported = 0; and, when the goal is full promotion, FAST capability = selected/selected. Keep the strict 5e-10 numerical gate and all existing model-row, runtime.score, restart-equivalence, and no-normal-update-file-I/O checks. Never weaken tolerances or remove parity dimensions to make a provider pass.

15A.4 Promote the staged certificate atomically and fail closed

Validate the staged JSON first, back up the current fixed production certificate, atomically copy the exact staged artifact to portfolio_live\outputs\phase_d4_provider_real_history_certification_v2.json, verify the promoted SHA matches the staging SHA, then run D4 static_overlay_prerequisites() and validate_activation(). If any post-release check fails, restore and byte-verify the previous production certificate. Never hand-edit certificate status/count fields.

> **Activation does not need certificate re-binding**
> The existing FAST_CONTINUATION activation is structural. If validate_activation() remains PASS after promotion, leave the activation file unchanged. On Moneta, the production certificate promotion moved the partition from 74 FAST / 29 AUTHORITATIVE to 103 FAST / 0 AUTHORITATIVE with the activation file unchanged.

15A.5 Run the focused overlay/router/registry closure

```
$env:PYTHONPATH = (Get-Location).Path

python .\tests\test_phase_d_04_provider_overlay.py
if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }

python .\tests\test_phase_d_scalable_router.py
if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }

python .\tests\test_phase_d_candidate_provider_registry.py
if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }

python .\tests\test_phase_d_primary_side_provider_registry.py
if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }
```

The overlay suite must prove current structural prerequisites, activation behavior, and explicit FAST failure -> AUTHORITATIVE_BATCH transition. The scalable-router suite must prove no hidden fallback. Both provider registries must retain dynamic exact-signature binding and fail-closed behavior.

- If the certifier was intentionally changed and the exact-byte integrity test is the only failure, verify the certifier SHA against the approved changed bytes and re-pin the test to that exact SHA. Do not remove the pin and do not compute the expected value dynamically from the file being tested.

15A.6 Run the final offline production-hardening regression gate

```
python .\portfolio_live\validate_production_hardening_patch.py
```

Required result: top-level status PASS. This validator is offline and uses temporary/fake MT5 fixtures; its synthetic execution subtests are not a substitute for the fresh dashboard/runtime checks in Part 17. Do not rerun the expensive consolidated D4 certificate merely because this final regression gate passes or fails for an unrelated production-hardening reason.

> **Runtime fail-safe remains authoritative**
> A production FAST provider exception must be visible, increment D_FAST_PATH_FAILURE, explicitly transition the affected edge to AUTHORITATIVE_BATCH, and use the authoritative generator thereafter. D4 acceleration must never create an unsupported or hidden execution path.

16. Verify FFD/HMM Live Feature Continuation

First perform a fresh health check without automatically modifying continuation state:

```
python .\portfolio_live\repair_live_feature_continuation.py --no-cache
```

If PASS: do not rebuild unnecessarily. If FAIL/BLOCKED:

```
python .\portfolio_live\repair_live_feature_continuation.py --repair --no-cache
python .\portfolio_live\validate_live_feature_continuation_replay.py
```

The repair option archives/rebuilds stale continuation outputs. --no-cache alone only forces a fresh check.

If this is NOT a new account (Section 16A does not apply) but the dashboard has not run in some time, expect this health check to fail closed for every asset with internal_ffd_coverage_error:required_file_missing — the coverage guard requires portfolio_live/live_bars/<ASSET>_live_standard_quote_tick_bars.csv (the dashboard's own rolling tick-bar output, separate from account/position/trade state) to already exist, and it has no broker-backfill capability of its own. That backfill only happens when the dashboard itself starts (Section 17, via run_startup_gap_healing()). For this case: proceed to Section 17 first to let the dashboard backfill live_bars from the broker, then re-run this Section 16 health check afterward — do not attempt --repair before Section 17 in this scenario, it will only fail and roll back.

16A. Archive Prior-Account Runtime State Before the First New-Account Startup

For a genuinely new prop account/broker deployment, do not carry the previous account's mutable dashboard/executor state or output ledgers into the first startup. Archive the primary state and its .last_good backup together. Deleting only the primary file is unsafe because production_state_integrity may legitimately recover from .last_good.

```
$stamp = Get-Date -Format "yyyyMMdd_HHmmss"
$archive = ".\portfolio_live\migration_archives\runtime_state_pre_new_account_$stamp"
New-Item -ItemType Directory -Force $archive | Out-Null

$paths = @(
  ".\portfolio_live\live_state\portfolio_live_dashboard_orchestrator_state.json",
  ".\portfolio_live\live_state\portfolio_live_dashboard_orchestrator_state.json.last_good",
  ".\portfolio_live\live_state\portfolio_execution_bot_state.json",
  ".\portfolio_live\live_state\portfolio_execution_bot_state.json.last_good",
  ".\portfolio_live\outputs\live_dashboard",
  ".\portfolio_live\outputs\portfolio_execution_bot"
)

foreach ($p in $paths) {
  if (Test-Path $p) {
    Move-Item $p (Join-Path $archive ([IO.Path]::GetFileName($p))) -Force
  }
}
"Runtime archive: $archive"
```

Why the output directories matter: old closed-trade CSV/history can pollute reconciliation and old dashboard JSONL can leave signals that are not part of the new-account process. Preserve immutable certificates/config/models/bar policy/research data; archive only mutable runtime state and runtime outputs.

> **If the new account has already been started**
> Do not blindly archive current files. First inspect the actual broker open positions and the current Quantara state/trade records so you do not erase evidence from the new account. Reconcile first, then archive only the state that is genuinely from the previous account or an intentionally discarded dry-run.

17. Start the Dashboard ALONE and Verify a Fresh Status

> **Executor still disarmed**
> At this point portfolio_execution_mode must still be paper and enable_order_send must still be false. Do not start an order-sending executor yet.
>
> Verify this directly: read portfolio_live/local_live_settings.json's live_mode, allow_real_live_trading_export, enable_order_send, and portfolio_execution_mode fields explicitly before this step — do not assume they are still at safe defaults. This file has been found pre-armed (enable_order_send=true, portfolio_execution_mode="live") at a point in the sequence well before Section 18's intended arming step; confirm current values match your actual intent for this run before proceeding, whichever way that should be.

After the runtime-state archive in Part 16A (when applicable), start the dashboard using your normal production dashboard command. After it has completed startup and written a new status, open:

```
portfolio_live\outputs\live_dashboard\latest_dashboard_status.json
```

A first startup attempt outside trading hours (weekend, or before the relevant market(s) reopen) will correctly fail closed at resolve_broker_clock_consensus with clock_contract_no_fresh_symbol_samples — the broker-clock lock requires fresh tick samples, and stale weekend/closed-market ticks do not qualify. The dashboard exits cleanly on its own (no persistent process, no state corruption, no order activity) rather than hanging or guessing a clock reference. This is expected fail-closed behavior, not a fault to debug — everything up through MT5 connection, edge artifact preload, and C4/D3/D4 routing installation can and should be verified clean from the log even when this final gate blocks on stale ticks; simply retry once there is live tick flow.

Verify the file timestamp/content is fresh for this startup, then confirm:

[ ] deployment/policy = PASS
[ ] selected asset count matches current manifest
[ ] selected edge count matches current manifest
[ ] broker clock = locked / PASS
[ ] continuity = PASS or PASS_HEALED_CONTINUOUS
[ ] quarantined assets = 0
[ ] recovery active = 0 and pending = 0
[ ] Phase C4 router = active
[ ] Phase D3 = PASS/active if hybrid D3 was enabled; otherwise intentional C4-only mode
[ ] Phase D4 = PASS/active if D3/D4 hybrid was enabled
[ ] unsupported trading edges = 0
[ ] D_FAST_PATH_FAILURE = 0 at startup/preflight

> **Why the old order was wrong**
> A pre-existing latest_dashboard_status.json can describe a previous process/configuration. You must start the dashboard first, then evaluate the fresh status generated by the current activation/account/feed state.

18. Final Executor Arming

Only after the dashboard-only preflight is clean should you change the local settings to actual order execution:

```
{
  "portfolio_execution_mode": "live",
  "enable_order_send": true
}
```

Then start/restart the executor. Do not change LIVE_MODE, account rules, symbol mappings, risk caps, thresholds, or activations between the dashboard preflight and executor startup unless you repeat the relevant validation.

19. Current Moneta Migration: Verified 22 August 2026 State

Based on the completed 21-22 August 2026 Moneta migration and D4 promotion evidence:

- Current structural universe: 10 selected assets and 103 selected edges. D1 current inventory PASS; D2 and D3 parity certifications PASS for 103/103; C4/D3 production routing remains valid.
- Provider coverage after implementation: 24 dynamically discovered candidate signatures and 24 primary-side signatures. The five newly implemented signature families covered the remaining seven provider-missing edges; the other 22 previously AUTHORITATIVE edges already had providers and needed fresh edge-level real-history certification only.
- Staged consolidated D4 real-history certificate: PASS, 103 edges checked, 103 passed, 0 failed, FAST capability 103/103, unsupported 0. The run retained the strict 5e-10 parity gate plus model-row, runtime.score, restart-equivalence, and normal-update no-file-I/O checks. Production was not changed by the staging run.
- Production D4 certificate promotion: PASS. Previous production SHA256 922fec9e3a3a62ae7217ec7a24fed5d6cd52f4190972b4aa160d2b4d794efc94; promoted certificate SHA256 f5d060c6b1bbb5048c2789390004d85be8c8bd2f8fbd13a983c864f0c0163142. A pre-103 production backup was written before replacement.

- Post-promotion D4 structural state: selected 103, FAST_CONTINUATION 103, AUTHORITATIVE_BATCH 0, unsupported 0; static prerequisites PASS; activation validation PASS; activation mode FAST_CONTINUATION. The D4 activation file was not changed because its structural contract remained valid.
- Focused D4 closure: 27/27 tests passed across test_phase_d_04_provider_overlay.py (10), test_phase_d_scalable_router.py (5), candidate provider registry (6), and primary-side provider registry (6). The certifier integrity pin was deliberately re-based only after the approved certifier SHA256 8807383df837f04901037360209e4bfd67e8da4c7fb124bdb6fcf809832d7acd was verified.
- Final offline production-hardening regression on 22 August 2026: top-level PASS, including broker-clock contract, P0, snapshot pipeline, state-integrity recovery, release/temp identity, execution fault-injection, and static-global-resolution checks. The validator uses fake/temporary fixtures; fixture fields such as execution_state_integrity_operator_recovery_required are not assertions about the actual live Moneta account.
- The AUTHORITATIVE-to-FAST engineering work is therefore closed. Remaining go-live gates are operational: fresh continuation health, prior-account runtime-state archive where applicable, dashboard-only fresh status with no quarantine/recovery/unsupported edges or D_FAST_PATH_FAILURE, and only then executor arming.

20. Stop / Pass Gates

age	Required result	Action
ocator 00A-11	Every required script exits 0; Script 11 = STRESS_PASS	**Stop**
e manifest 00-02	LIVE_APPROVED, live_ready=True, live_trade_enabled=True, environment PASS	**Stop**
r policy 03-04	All selected assets PASS; POLICY_PASS	**Stop**
anged-universe C1/C2/C3	All current-universe authorities PASS	**Stop**
/ D4.1 current inventory	Counts match current live manifest; PASS	**Stop**
e-C5 exact test gate	Focused validators PASS AND exact full unittest discovery exits 0 / ends OK	Stop
	All nine required gates PASS; production certificate written	**Stop**
activation	Valid sealed C5 certificate; activation written	**Stop**
/D4 hybrid	Activation validations PASS; unsupported=0. AUTHORITATIVE coverage is valid unless a separate FAST promotion is deliberately performed.	Stop on invalid authority; do not for FAST
tional D4 FAST promotion	If performed: staged full selected-edge cert PASS; atomic production promotion; focused D4 closure PASS	Stop / rollback on any mismatch
nal hardening regression	validate_production_hardening_patch.py top-level PASS	Stop
or-account runtime state	Primary + .last_good + old dashboard/executor outputs archived together, or explicitly reconciled if new account already started	Stop
ntinuation	Fresh PASS, or repair + replay PASS	**Stop**
shboard-only startup	Fresh current status clean; no quarantine/recovery/unsupported edges	**Stop**
ecutor arming	Only after every previous gate	**Go live**

21. One-Page Correct Sequence

1. Stop dashboard + executor.
2. Configure active_account_rules.json from the real prop rules.
3. Configure local_live_settings.json with real account/symbol mapping but execution kept paper / order_send false.
4. Connect intended MT5 account/server; update broker symbol seed if needed.
5. Run allocator 00A -> 11; require STRESS_PASS.
6. Build/validate live manifest 00 -> 02.
7. New broker/feed: run fresh bar diagnostic 03 -> freeze 04.
8. Compare new live selected edges with existing C3 authority.

9A. If unchanged and C4 read-only check passes: keep Phase C authority; validate current hybrid routing.

9B. If changed/stale: back up/remove C4 + D3 + D4 activations.

10. Rebuild C1/C2/C3 current-universe authorities.

11. Refresh D1 current selected-edge inventory BEFORE C5.

12. Refresh inert D4.1 contract.

13. Verify signal-age contract; run focused hardening/P0/D checks.

14. Run the exact full unittest discovery used by C5; require OK. Fix stale assertions/import-time negative controls before C5; then seal and validate C5.

15. Activate C4.

16. Generate/validate current D3 hybrid activation only from valid certified coverage.

17. Validate the D4 overlay activation and current hybrid partition; require unsupported_trading_edges = 0.

18. Optional acceleration only: inventory missing D4 coverage, implement exact-signature providers, run a staged full consolidated real-history certificate, atomically promote the certificate, then run the focused D4 closure. Do not change alpha/research logic.

19. Run the final offline production-hardening validator; require top-level PASS.

20. Fresh FFD/HMM continuation health; repair only if blocked.

21. For a new account, archive previous primary + .last_good dashboard/executor state and old runtime output directories together. If the new account already ran, reconcile before archiving.

22. Start dashboard only; verify a fresh current status with no quarantine/recovery/unsupported edges and no D_FAST_PATH_FAILURE.

23. Only then switch portfolio_execution_mode=live and enable_order_send=true.

24. Start/restart the executor and monitor the fresh account state.

Appendix A. Repository Source Basis for This Correction

The ordering and interpretations above were checked against the current v2 implementations below. These are code-path references, not generic trading-system assumptions.

portfolio_live/phase_c_05_certify_production_routing.py - C5 requires C4 activation absent; runs C1/C2/C3 validation, all-edge parity, P0, hardening, and `python -m unittest discover -s tests -p test_*.py -v`. The runbook therefore executes that exact full-suite gate before C5.

portfolio_live/phase_c_04_prepare_activation.py - Read-only by default; explicit activation requires the acknowledgement flag and a valid sealed C5 certificate.

portfolio_live/phase_d_01_compile_candidate_contract.py - D1 is an offline current selected-edge inventory/contract compiler and never enables routing.

portfolio_live/phase_d_03_production_router.py - Hybrid D3 treats retained D2/D3-certified current edges as optimized and the remaining current selected edges as C4-authoritative; unsupported=0; activation absent is inert.

portfolio_live/phase_d_04_provider_overlay.py - Hybrid D4 partitions current selected edges into FAST_CONTINUATION and AUTHORITATIVE_BATCH; edge-level real-history certification is required for FAST only.

portfolio_live/phase_d_04_compile_incremental_candidate_contract.py - Inert D4.1 current source-signature contract compiler.

tests/test_phase_d_candidate_first.py - Requires D1/C3 source guard to match current edge universe.

tests/test_phase_d_incremental_candidate_contract.py - Requires current D4.1 compilation to cover current D1/live-manifest universe.

portfolio_live/validate_production_hardening_patch.py - Enforces the current dashboard/executor signal-validity production contract; if it reports PASS, no signal-age edit is required.

tests/test_phase_d_04_provider_overlay.py - Enforces hybrid-safe complete/disjoint FAST + AUTHORITATIVE coverage, activation inertness, shadow identity checks, and explicit FAST failure -> authoritative fallback. It also contains an intentional exact-SHA pin for the consolidated certifier; re-pin only after verifying approved changed bytes.

tests/test_phase_d_volatilitycarry_candidate_negative_control.py - Intentional must-fail parity control. Its executable mutation/assertion must remain import-safe under unittest discovery while standalone execution remains non-zero.

portfolio_live/00_build_live_deployment_manifest.py - REAL_LIVE + allow export gates live-approved manifest, separately from executor order-send arming.

portfolio_live/02_validate_live_environment.py - Checks current account rules, selected runtimes, symbol mappings, and explicit per-asset execution guards.

portfolio_live/repair_live_feature_continuation.py - --no-cache performs a fresh check; --repair is what archives/rebuilds stale continuation state.

portfolio_live/certification/phase_d4_consolidated_real_history_certifier_v3.py - Full edge-level real-history D4 parity authority. Use a separate staging output for promotion work; preserve strict tolerance/model/score/restart/no-I/O gates.

shared/edges/candidate_continuations/registry.py and shared/edges/composed_decision_continuations primary registry - Dynamic provider discovery keyed by exact logic signatures; duplicate signature claims fail closed; no fixed asset registry.

tests/test_phase_d_scalable_router.py - Verifies FAST capability classification, explicit transition to AUTHORITATIVE_BATCH, and no hidden authoritative fallback.

tests/test_phase_d_candidate_provider_registry.py and tests/test_phase_d_primary_side_provider_registry.py - Verify dynamic exact-signature provider binding and fail-closed registry behavior.

portfolio_live/production_state_integrity.py - .last_good recovery means new-account state migration must archive primary and backup state together, not delete only the primary file.

Appendix B. Phase-D Mental Model

Think of the production stack as nested authority, not as a single all-or-nothing certification:

```
C4 AUTHORITATIVE ROUTING  (must cover every current selected edge)
  |
  +-- D3 optimized subset  (only currently selected edges retained by D2/D3 certs)
       |
       +-- D4 FAST_CONTINUATION subset (only edge-level D4-certified/provider-capable edges)
       |
       +-- AUTHORITATIVE path for uncovered / transitioned edges

Current edge not D2/D3-certified?  -> C4 authority.
Current D3 edge not D4-certified?  -> authoritative candidate path.
FAST provider failure at runtime?  -> explicit transition to AUTHORITATIVE_BATCH.
Unsupported trading edges?         -> must remain 0.
```

> **The consequence**
> A new broker or a larger selected universe does not justify recertifying performance optimizations merely to make the deployment legal. First restore complete C4 authority. Then reuse only acceleration evidence that still applies. If you later promote every selected edge to D4 FAST, the authoritative path is still retained as the fail-safe runtime transition target; "103 FAST" does not mean the authoritative implementation has been removed.

Appendix C. Final Go-Live Checklist

[ ] Correct MT5 account/server connected.

[ ] active_account_rules.json reflects actual current prop rules and real current balance/HWM.

[ ] local_live_settings.json contains target credentials/symbol mapping and is not tracked by Git.

[ ] Allocator 00A-11 complete; Script 11 STRESS_PASS.

[ ] Live manifest/environment PASS and current.

[ ] Fresh broker bar policy PASS.

[ ] Current C1/C2/C3 authority valid for selected universe.

[ ] D1 current inventory equals live manifest.

[ ] Any intentional negative-control test_*.py modules are import-safe; standalone must-fail controls still fail when run directly.

[ ] Exact full unittest discovery exits 0 and ends OK before C5.

[ ] C5 certificate PASS and current.

[ ] C4 activation valid.

[ ] D3/D4 activations either valid hybrid subset or intentionally absent for C4-only mode.

[ ] unsupported_trading_edges = 0.

[ ] If optional D4 FAST promotion was performed: full staged selected-edge certificate PASS; production certificate promoted with backup/rollback guard; current partition validated; focused D4 tests PASS.

[ ] Final offline production-hardening validator top-level PASS after the latest D4/test changes.

[ ] Previous-account dashboard/executor primary + .last_good state and old runtime outputs archived together, or explicitly reconciled if the target account was already started.

[ ] FFD/HMM continuation healthy.

[ ] Dashboard-only fresh startup PASS with no quarantine/recovery blockers.

[ ] No D_FAST_PATH_FAILURE at preflight.

[ ] Only now: portfolio_execution_mode=live and enable_order_send=true.

[ ] Executor started after dashboard verification.

22. Appendix: 2026-09-26 Live Redeploy Cycle — Runbook Corrections & Findings

This appendix records gaps between this runbook and what was actually encountered certifying the calendar-time-relabeled 12-edge universe (DAX, GBPJPY, HK, JPN, US500, XAUUSD) for live routing on the MonetaFunded-Live account, including a full Section 15A AUTHORITATIVE-to-FAST provider promotion. Each item below is a correction or addition to the numbered sections above, not a new procedure.

22.1 C4/D3 activation backup filename collision

portfolio_live/config/phase_c/production_routing_activation.json and portfolio_live/config/phase_d/production_routing_activation.json share the same basename (production_routing_activation.json). A backup script that names backups by basename alone (including this runbook's own PowerShell backup snippet) will silently overwrite the C4 backup with the D3 backup if both are backed up to the same directory in one pass. Always include the phase_c/ or phase_d/ path segment (or the full relative path) in any backup filename for these two files, never the basename alone.

22.2 D2/D3 stale-certification trap (Section 14)

phase_d_03_production_router.py's static_contract_report()/_phase_d310_d2_certified_edge_ids() classify an edge as D2/D3-optimized purely by matching edge_id STRING against candidate_first_certification.json's edges dict with status=PASS -- there is no candidate_logic_signature (or any other source-code fingerprint) cross-check against the CURRENT D1 contract. After the 2026-09-25 calendar-time relabel, running Section 14.2 against 12-edge-unchanged-name/changed-logic edges produced a suspiciously clean '12 optimized, 0 authoritative' result -- i.e. it silently credited ALL 12 edges with a stale pre-relabel D2/D3 certification purely because their edge_id strings were unchanged.

Correction: before trusting a D2/D3 'optimized' classification after ANY change to an edge's candidate/primary logic source, manually confirm the D2/D3 certification file mtimes are AFTER that change, or simply rerun phase_d_02_certify_candidate_first.py and phase_d_03_certify_production_router.py fresh (see 22.5 for the required venv). The documented safe fallback -- delete the D3 activation and proceed with D3 ABSENT (pure C4-authoritative routing), unsupported_trading_edges=0 -- remains valid whenever you cannot immediately re-certify, but see 22.4: this fallback blocks Section 15A.

22.3 Section 15A.2 scope correction: port ALL providers on a geometry change

Section 15A.2 as written says to 'implement only the missing exact-signature provider families.' After a labeling/geometry change that touches EVERY edge's Event_Vol computation (such as the 2026-09-24 bar-count-to-calendar-time rewrite), this is wrong: every EXISTING provider that computes Event_Vol also needs its lag-lookup mechanism ported, not just edges that were previously unbound. All 9 existing candidate-side provider files needed the bar-count-to-calendar-time bisect-based port this cycle, not a handful of missing ones. Verify each ported provider against the shared reference function (shared/labels/calendar_time_barriers.py:calendar_time_event_vol) with a synthetic parity test before certifying.

22.4 Section 15A depends on an active D3 layer

phase_d4_consolidated_real_history_certifier_v3.py (Section 15A.3) hard-requires portfolio_live/config/phase_d/production_routing_activation.json to exist AND requires the D3 certification to cover 100% of currently-selected edges with PASS (d3_chain_ok). portfolio_live/certification/README.md confirms

FAST_CONTINUATION mode feeds provider events into 'the existing D.3 downstream scorer' -- the FAST overlay is architecturally built on top of an active D3 layer, not a standalone replacement for C4-only routing.

Correction: if you followed 22.2's safe fallback and are running with D3 ABSENT (pure C4-authoritative), Section 15A cannot proceed until D2 and D3 are legitimately re-certified fresh for the full current edge universe and the D3 activation is sealed again (phase_d_03_certify_production_router.py --seal-production). Budget for this before attempting 15A, it is materially more work than the provider-porting step alone.

22.5 Phase C/D scripts must run under venv39, not venv312

shared/edges/phase_c_feature_contract.py's _runtime_build_features_ast_hash() hashes ast.dump(node, annotate_fields=True, include_attributes=False) of each edge's build_features() function. Python 3.12 emits an extra type_params=[] field on FunctionDef (PEP 695) that Python 3.9 does not. Running any phase_c_0X_*.py or phase_d_0X_*.py script under venv312 (Python 3.12) instead of venv39 (Python 3.9, the environment the live frozen contracts were actually built under) therefore produces a spurious 'feature contract structural asset DAG state changed' FAIL for every edge uniformly, even when nothing in the actual source changed.

Correction: venv312 is correct for the Step01-09 edge-lifecycle research scripts. ALWAYS use venv39 for phase_c_01 through phase_c_05 and phase_d_01 through phase_d_04, including the Section 15A certifier and any ad-hoc diagnostic recompilation of the feature/candidate contracts.

22.6 Section 9 (bar policy rebuild) applies to ANY universe change, not only new-broker migrations

Section 9's heading says 'New Broker / Feed: Rebuild the Live Bar Policy,' but validate_live_inputs()'s policy_structurally_matches_manifest() check compares live_bar_policy.json's frozen source_manifest_created_at and edge/asset lists against the CURRENT live_manifest.json unconditionally -- it does not care why the manifest changed, only that it changed. Rebuilding the live manifest in Section 8 for a selected-edge-universe change (without a broker/feed change) leaves live_bar_policy.json stale, and the dashboard will crash at startup with 'Frozen live bar policy does not structurally match the current live manifest' the first time Section 17 is attempted.

Correction: rerun 03_live_bar_threshold_diagnostic.py --fresh and 04_freeze_live_bar_thresholds.py --require-pass (Section 9) any time Section 8 produces a new live_manifest.json with a changed selected-edge or selected-asset set, regardless of whether the broker/feed itself changed.

22.7 Section 17 broker-clock guard: a second named expected fail-closed variant

Section 17 already documents that a first startup outside trading hours (weekend, or before the relevant market reopens) correctly fails closed at resolve_broker_clock_consensus with clock_contract_no_fresh_symbol_samples. A Saturday dashboard-only startup attempt (12 edges, 6 assets, otherwise fully clean: MT5 connected, all edge artifacts preloaded, D4 FAST_CONTINUATION overlay active) instead raised BrokerClockContractError: clock_transition_mixed_old_new_samples, because the last-known ticks across the 6 symbols disagreed on session offset during the weekend close (previous_offset vs a mixed set of candidate offsets across symbols).

Correction: treat clock_transition_mixed_old_new_samples as a second named example of the same expected weekend/market-closed fail-closed category as clock_contract_no_fresh_symbol_samples -- not a new bug to debug. Everything up through MT5 connection, edge artifact preload, and C4/D3/D4 routing installation should still be verified clean from the log even when this gate blocks; simply retry once there is live tick flow.